MySQL Assignment 1 - DDL Commands, Constraints

Objective:

The objective of this project is to design and manage an Employee Database using MySQL Data Definition Language (DDL) commands. The project aims to create and maintain database objects such as databases and tables, establish relationships between employees, departments, and locations using primary and foreign keys, and apply appropriate constraints to ensure data integrity, consistency, and accuracy. Additionally, the project demonstrates the use of DDL operations including CREATE, ALTER, RENAME, TRUNCATE, and DROP for effective database management.

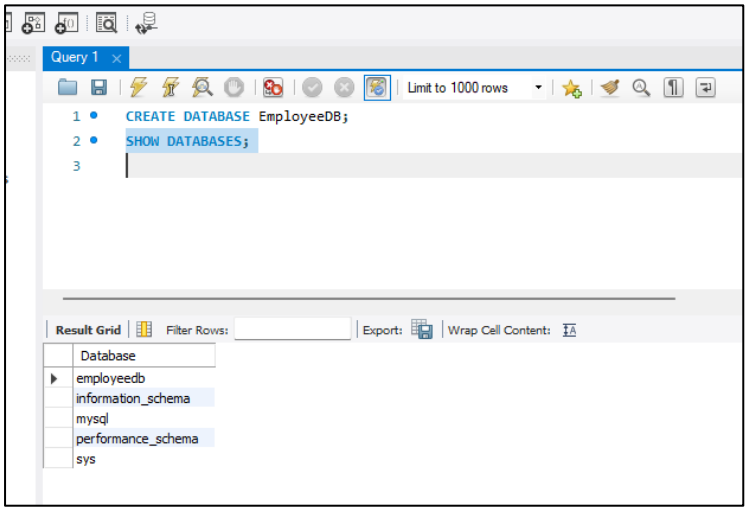
Tasks:

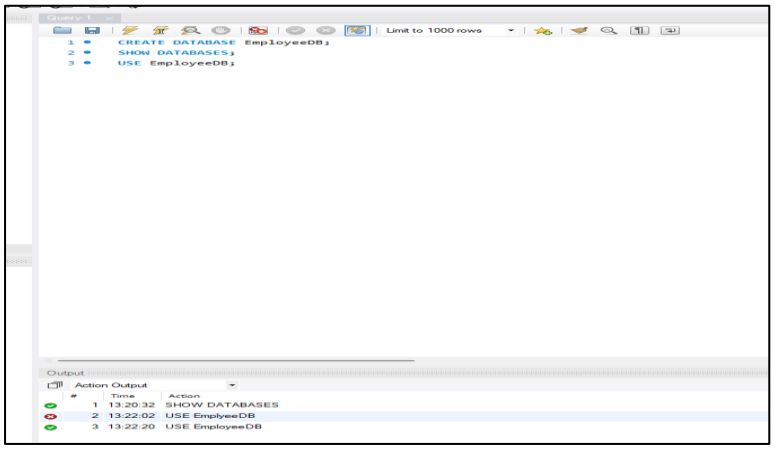
DDL Commands

1. Database and Table Creation (CREATE):

- Create a Database for this project
- Create all three tables in MySQL with appropriate data types and relationships.

Create Database:





Create Tables:

Table: Departments

Attributes:

○ department_id – Primary Key

○ department_name

```
USE EmployeeDB;
CREATE TABLE Departments(
department_id INT PRIMARY KEY,
department_name VARCHAR(50)
);
```

```
5  13:27:41  CREATE TABLE Departments( department_id INT PRIMARY KEY, department_name VARCHAR(50) )  0 row(s) affected
```

Table: Location

Attributes:

○ location_id – Primary Key

○ location_name

```
);
CREATE TABLE Location(
location_id INT PRIMARY KEY,
location_name VARCHAR(50)
);
```

```
7  13:30:03  CREATE TABLE Location( location_id INT PRIMARY KEY, location_name VARCHAR(50) )  0 row(s) affected
```

Table: Employees

Attributes:

○ employee_id – Primary Key

○ Employee_name

○ Gender

○ Age

○ Hire_date

○ Designation

○ Salary

○ department_id – Foreign Key referencing department(department_id)

○ location_id—Foreign Key referencing location(location_id)

```
);
CREATE TABLE Employees(
employee_id INT PRIMARY kEY,
Employee_name VARCHAR(100),
Gender ENUM('M','F'),
Age INT,
Hire_date DATE,
Designation VARCHAR(100),
Salary DECIMAL(10,2),
department_id INT,
location_id INT
);
```

```
8  13:37:03  CREATE TABLE Employees( employee_id INT PRIMARY kEY, Employee_name VARCHAR(100), Gender ENUM('M','F'), Age INT, Hire_date DATE, D...  0 row(s) affected
```

2. Table Alteration (ALTER):

● Add a new column named "email" to the Employees table to store employee email addresses.

```
26      );
27 •    ALTER TABLE Employees
28      ADD email VARCHAR(50);
```

```
11  13:53:13  ALTER TABLE Employees ADD email VARCHAR(50)  0 row(s) affected Records: 0  Duplicates: 0  Warnings: 0
```

● Modify the data type of the "designation" column in the Employees table to support a wider range of values.

```
29 •    ALTER TABLE Employees
30      MODIFY Designation VARCHAR(200);
```

```
12  13:57:22  ALTER TABLE Employees MODIFY Designation VARCHAR(200)  0 row(s) affected Records: 0  Duplicates: 0  Warnings: 0
```

```
32 •  DESCRIBE Employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
employee_id	int	NO	PRI	NULL	
Employee_name	varchar(100)	YES		NULL	
Gender	enum('M','F')	YES		NULL	
Age	int	YES		NULL	
Hire_date	date	YES		NULL	
Designation	varchar(200)	YES		NULL	
Salary	decimal(10,2)	YES		NULL	
department_id	int	YES		NULL	
location_id	int	YES		NULL	
email	varchar(50)	YES		NULL	

● Drop the “age” column from the Employees table.

```
33 •  ALTER TABLE Employees
34    DROP COLUMN age;
35 •  DESCRIBE Employees;      /* checking age drop query */
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
employee_id	int	NO	PRI	NULL	
Employee_name	varchar(100)	YES		NULL	
Gender	enum('M','F')	YES		NULL	
Hire_date	date	YES		NULL	
Designation	varchar(200)	YES		NULL	
Salary	decimal(10,2)	YES		NULL	
department_id	int	YES		NULL	
location_id	int	YES		NULL	
email	varchar(50)	YES		NULL	

```
16  14:04:35  ALTER TABLE Employees DROP COLUMN age        0 row(s) affected Records: 0  Duplicates: 0  Warnings: 0
```

● Rename the “hire_date” column to “date_of_joining”.

```
36 •  ALTER TABLE Employees
37    RENAME COLUMN Hire_date TO date_of_joining;
38 •  DESCRIBE Employees;      /* checking rename query */
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
employee_id	int	NO	PRI	NULL	
Employee_name	varchar(100)	YES		NULL	
Gender	enum('M','F')	YES		NULL	
date_of_joining	date	YES		NULL	
Designation	varchar(200)	YES		NULL	
Salary	decimal(10,2)	YES		NULL	
department_id	int	YES		NULL	
location_id	int	YES		NULL	
email	varchar(50)	YES		NULL	

```
18  14:08:39  ALTER TABLE Employees RENAME COLUMN Hire_date TO date_of_joining        0 row(s) affected Records: 0  Duplicates: 0  Warnings: 0
```

3. Table Renaming (RENAME):

● Rename the "Departments" table to "Departments_Info".

```
39 •   RENAME TABLE Departments TO Departments_Info;  /* Renaming Department table name */
```

```
21  14:14:28  RENAME TABLE Departments TO Departments_Info                    0 row(s) affected
```

● Rename the "Location" table to "Locations".

```
40 •   RENAME TABLE Location TO Locations;  /* Renaming Location table name */
```

```
22  14:16:44  RENAME TABLE Location TO Locations                    0 row(s) affected
```

4. Table Truncation (TRUNCATE):

● Truncate the Employees table.

```
41 •   TRUNCATE Table Employees;
```

```
24  14:19:03  TRUNCATE Table Employees                    0 row(s) affected
```

```
41 •   TRUNCATE Table Employees;
42 •   SELECT * FROM Employees;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

employee_id	Employee_name	Gender	date_of_joining	Designation	Salary	department_id	location_id	email
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

5. Database & Table Dropping (DROP):

● Drop the Employees table and then the “employee” database.

```
43 •   DROP TABLE Employees;
44 •   DROP DATABASE EmployeeDB;
```

```
27  14:25:37  DROP TABLE Employees                    0 row(s) affected
28  14:26:16  DROP DATABASE EmployeeDB                    3 row(s) affected
```

```
43 •  DROP TABLE Employees;
44 •  DROP DATABASE EmployeeDB;
45 •  SHOW DATABASES;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Database
information_schema
mysql
performance_schema
sys

Constraints

1. Database Recreation:

● Drop the 'employee' database if it exists

● Recreate it using the provided schema, ensuring that all tables are created with the appropriate constraints as instructed.

● Establish links between the "department_id" and "location_id" fields in the "employees" table and their respective tables.

```
46 •  DROP DATABASE IF EXISTS EmployeeDB;
```

```
31  14:31:48  DROP DATABASE IF EXISTS EmployeeDB    0 row(s) affected, 1 warning(s): 1008 Can't drop database 'employeedb'; database doesn't exist
```

```
47 •  CREATE DATABASE EmployeeDB;
48 •  USE EmployeeDB;
```

```
32  14:35:07  CREATE DATABASE EmployeeDB    1 row(s) affected
33  14:35:27  USE EmployeeDB                0 row(s) affected
```

```
49 •  SHOW DATABASES;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Database
employeedb
information_schema
mysql
performance_schema
sys

2. Departments Table:

● Ensure that the "department_id" uniquely identifies each department.

● Set up constraints on the "department_name" to avoid duplicate and null entries.

```
50 • CREATE TABLE Departments(
51   department_id INT PRIMARY KEY,
52   department_name VARCHAR(50) NOT NULL UNIQUE
53   );
```

```
35  16:44:17  CREATE TABLE Departments( department_id INT PRIMARY KEY, department_name VARCHAR(50) NOT NULL UNIQUE )  0 row(s) affected
```

```
54 • DESCRIBE Departments;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
department_id	int	NO	PRI	NULL	
department_name	varchar(50)	NO	UNI	NULL	

3. Locations Table:

● Establish a mechanism to automatically generate unique identifiers for each location, ensuring that they are incremented sequentially.

● Implement constraints to prevent the insertion of null and duplicate locations.

```
55 • CREATE TABLE Locations(
56   location_id INT PRIMARY KEY auto_increment,
57   location_name VARCHAR(50) NOT NULL UNIQUE
58   );
```

```
38  16:58:31  CREATE TABLE Locations( location_id INT PRIMARY KEY auto_increment, location_name VARCHAR(50) NOT NULL UNIQUE )  0 row(s) affected
```

```
59 • DESCRIBE Locations;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
location_id	int	NO	PRI	NULL	auto_increment
location_name	varchar(50)	NO	UNI	NULL	

4. Employees Table:

- Guarantee that each employee has a distinct identifier.
- Create a restriction to ensure that the employee's name is always provided.
- Limit the acceptable values for the "gender" field to only 'M' or 'F'.
- Enforce a condition to ensure that the employee's age is 18 or above.
- Automatically assign the current date to the "hire_date" field if not specified.

```
60  CREATE TABLE Employees(
61  employee_id INT PRIMARY KEY,
62  Employee_name VARCHAR(100) NOT NULL,
63  Gender ENUM('M','F') NOT NULL,
64  Age INT CHECK(Age >= 18),
65  Hire_date DATE DEFAULT(CURRENT_DATE),
66  Designation VARCHAR(100),
67  Salary DECIMAL(10,2),
68  department_id INT,
69  location_id INT,
70  FOREIGN KEY (department_id)
71  REFERENCES Departments(department_id),
72  FOREIGN KEY (location_id)
73  REFERENCES Locations(location_id)
74  );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
employee_id	int	NO	PRI	NULL	
Employee_name	varchar(100)	NO		NULL	
Gender	enum('M','F')	NO		NULL	
Age	int	YES		NULL	
Hire_date	date	YES		curdate()	DEFAULT_GENERATED
Designation	varchar(100)	YES		NULL	
Salary	decimal(10,2)	YES		NULL	
department_id	int	YES	MUL	NULL	
location_id	int	YES	MUL	NULL	

```
43  17:12:36  CREATE TABLE Employees( employee_id INT PRIMARY KEY, Employee_name VARCHAR(100) NOT NULL, Gender ENUM('M','F') NOT NULL, Age I...  0 row(s) affected
44  17:13:07  DESCRIBE Employees                                                                                                              9 row(s) returned
```

```
76  SHOW TABLES;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

Tables_in_employeedb
departments
employees
locations

Conclusion:

This project successfully demonstrated the implementation of an Employee Database in MySQL using DDL commands. The database was created with appropriate tables, relationships, and constraints to ensure data integrity and consistency. Various database operations, including creating, modifying, renaming, truncating, and dropping database objects, were performed successfully. By applying primary keys, foreign keys, unique constraints, check constraints, and default values, the database was designed to maintain reliable and organized employee information. This project enhanced the understanding of database design principles and the practical use of MySQL DDL commands for efficient database management.